Research Notes on Encoder decoder architecture

Encoder decoder architecture

>> Section 1

machine translation time series prediction document summarization document generation named entity recognition (NER) writing computer code based on requirements expressed in natural language. speech-to-text Beyond traditional NLP, the transformer architecture has had success in other applications, such as:

>> Section 2

Speculative decoding is a method to accelerate token decoding. Similarly to speculative execution in CPUs, future tokens are computed quickly, then verified. If the quickly computed tokens are incorrect, they are discarded and computed slowly. The key factor in speculative decoding is that a transformer decoder can verify faster than it can decode, in the following sense. Suppose we have two transformer models like GPT-3 and GPT-3-small, both with a context window size of 512. To generate an entire context window autoregressively with greedy decoding with GPT-3, it must be run for 512 times, each time generating a token x 1 , x 2 , . . . , x 512 {\displaystyle x_{1},x_{2},...,x_{512}} , taking time 512 T GPT-3 {\displaystyle 512T_{\text{GPT-3}}} . However, if we had some educated guess for the values of these tokens, we could verify all of them in parallel, in one run of the model, by checking that each x t {\displaystyle x_{t}} is indeed the token with the largest log-likelihood in the t {\displaystyle t} -th output. In speculative decoding, a smaller model or some other simple heuristic is used to generate a few speculative tokens that are subsequently verified by the larger model. For example, suppose we use GPT-3-small to generate four speculative tokens: x ~ 1 , x ~ 2 , x ~ 3 , x ~ 4 {\displaystyle {\tilde {x}}_{1},{\tilde {x}}_{2},{\tilde {x}}_{3},{\tilde {x}}_{4}} . This only takes 4 T GPT-3-small {\displaystyle 4T_{\text{GPT-3-small}}} . These tokens are then run through the larger GPT-3 in one go. Suppose that x ~ 1 {\displaystyle {\tilde {x}}_{1}} and x ~ 2 {\displaystyle {\tilde {x}}_{2}} are verified by GPT-3 as what it would have picked, then those are kept, but x ~ 3 {\displaystyle {\tilde {x}}_{3}} is not, so x ~ 3 , x ~ 4 {\displaystyle {\tilde {x}}_{3},{\tilde {x}}_{4}} are discarded, and GPT-3 is run on those. This would take 4 T GPT-3-small + 3 T GPT-3 {\displaystyle 4T_{\text{GPT-3-small}}+3T_{\text{GPT-3}}} , which might be shorter than 4 T GPT-3 {\displaystyle 4T_{\text{GPT-3}}} . For non-greedy decoding, similar ideas apply, except the speculative tokens are accepted or rejected stochastically, in a way that guarantees the final output distribution is the same as if speculative decoding was not used.

>> Section 3

The feedforward network (FFN) modules in a transformer are 2-layered multilayer perceptrons: F F N (x) = φ (x W (1) + b (1)) W (2) + b (2) {\displaystyle \mathrm {FFN} (x)=\phi (xW^{(1)}+b^{(1)})W^{(2)}+b^{(2)}} where W (1) {\displaystyle W^{(1)}} and W (2) {\displaystyle W^{(2)}} are weight matrices and b (1) {\displaystyle b^{(1)}} and b (2) {\displaystyle b^{(2)}} are bias vectors, and φ {\displaystyle \phi } is its activation function. The original transformer used ReLU activation. The number of neurons in the middle layer is called intermediate size (GPT), filter size (BERT), or feedforward size (BERT). It is typically larger than the embedding size. For example, in both GPT-2 series and BERT series, the intermediate size of a model is 4 times its embedding size: d ffn = 4 d emb {\displaystyle d_{\text{ffn}}=4d_{\text{emb}}} .

>> Section 4

/* second sublayer */ z_e_copy ← copy(z_e) for each t in 1:length(z_e) do z_e[t] ← layer.layer_norm(z_e[t]) z_e ← layer.feedforward(z_e) for each t in 1:length(z_e) do z_e[t] ← z_e[t] + z_e_copy[t]

>> Section 5

Alternative normalizations The normalization used in the transformer can be different from LayerNorm. One example is RMSNorm which is used in the Llama series. Other examples include ScaleNorm and FixNorm.

>> Section 6

The final points of detail are the residual connections and layer normalization, (denoted as "LayerNorm", or "LN" in the following), which while conceptually unnecessary, are necessary for numerical stability and convergence. The residual connection, which is introduced to avoid vanishing gradient issues and stabilize the training process, can be expressed as follows: y = F(x) + x. The expression indicates that an output y is the sum of the transformation of input x (F(x)) and the input itself (x). Adding the input x can preserve the input information and avoid issues when the gradient of F(x) is close to zero. Similarly to how the feedforward network modules are applied individually to each vector, the LayerNorm is also applied individually to each vector. There are two common conventions in use: the post-LN and the pre-LN convention. In the post-LN convention, the output of each sublayer is L a y e r N o r m (x + S u b l a y e r (x)) {\displaystyle \mathrm {LayerNorm} (x+\mathrm {Sublayer} (x))} where S u b l a y e r (x) {\displaystyle \mathrm {Sublayer} (x)} is the function implemented by the sublayer itself. In the pre-LN convention, the output of each sublayer is x + S u b l a y e r (L a y e r N o r m (x)) {\displaystyle x+\mathrm {Sublayer} (\mathrm {LayerNorm} (x))} The original 2017 transformer used the post-LN convention. It was difficult to

train and required careful hyperparameter tuning and a "warm-up" in learning rate, where it starts small and gradually increases. The pre-LN convention, proposed several times in 2018, was found to be easier to train, requiring no warm-up, leading to faster convergence.

>> Section 7

/* third sublayer */ z_d_copy ← copy(z_d) for each t in 1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t]) z_d ← layer.feedforward(z_d) for each t in 1:length(z_d) do z_d[t] ← z_d[t] + z_d_copy[t]

>> Section 8

Predecessors For many years, sequence modelling and generation was done by using plain recurrent neural networks (RNNs). A well-cited early example was the Elman network (1990). In theory, the information from one token can propagate arbitrarily far down the sequence, but in practice the vanishing-gradient problem leaves the model's state at the end of a long sentence without precise, extractable information about preceding tokens. A key breakthrough was LSTM (originally described in a 1995 technical report and formally published in 1997), an RNN that introduced gating mechanisms to mitigate the vanishing gradient problem, allowing efficient learning of long-sequence modelling. One key architectural element was the use of multiplicative gating units, in which the outputs of some neurons modulate the outputs of others. These multiplicative units are conceptually distinct from the additive attention mechanism later introduced for sequence-to-sequence models. Neural networks using multiplicative units were later called sigma-pi networks or higher-order networks. LSTM became the standard architecture for long sequence modelling until the 2017 publication of transformers. However, LSTM still used sequential processing, like most other RNNs. Specifically, RNNs operate one token at a time from first to last; they cannot operate in parallel over all tokens in a sequence. Modern transformers overcome this problem, but unlike RNNs, they require computation time that is quadratic in the size of the context window. The linearly scaling fast weight controller (1992) learns to compute a weight matrix for further processing depending on the input. One of its two networks has "fast weights" or "dynamic links" (1981). A slow neural network learns by gradient descent to generate keys and values for computing the weight changes of the fast neural network which computes answers to queries. This was later shown to be equivalent to the unnormalized linear transformer.